Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	1	Academic Year	:	2026-27

Task - 1

Aim

To set up a React development environment using Vite and build a static student portfolio page composed of independently structured, reusable functional components (Header, About, Skills, and Footer), demonstrating component composition and one-way data flow from parent to child components through props.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-1.git

Key Architectural Components:

1. Header.jsx: Reusable top branding banner accepting 'name', 'themeColor', and 'subtitle' props.
2. About.jsx: Student biography, department, roll number, and contact links.
3. Skills.jsx: Dynamic technical skills listing rendering reusable SkillBadge components.
4. Footer.jsx: Standardized copyright and academic institution metadata footer.

```
// src/components/Header.jsx
function Header({ name = 'Param Shah', subtitle }) {
 return (
  <header className="portfolio-header">
   <h1>Student Portfolio & Full Stack Platform</h1>
   <p className="header-name">{name}</p>
   {subtitle && <p className="header-subtitle">{subtitle}</p>}
  </header>
 );
}
export default Header;

// src/components/Skills.jsx
function Skills({ skills }) {
 return (
  <section className="skills-grid">
   {skills.map((skill) => (
    <span key={skill.name} className="skill-badge" style={{ background: skill.color }}>
     {skill.name}
    </span>
   ))}
  </section>
```

```
  );
}
export default Skills;
```

Output

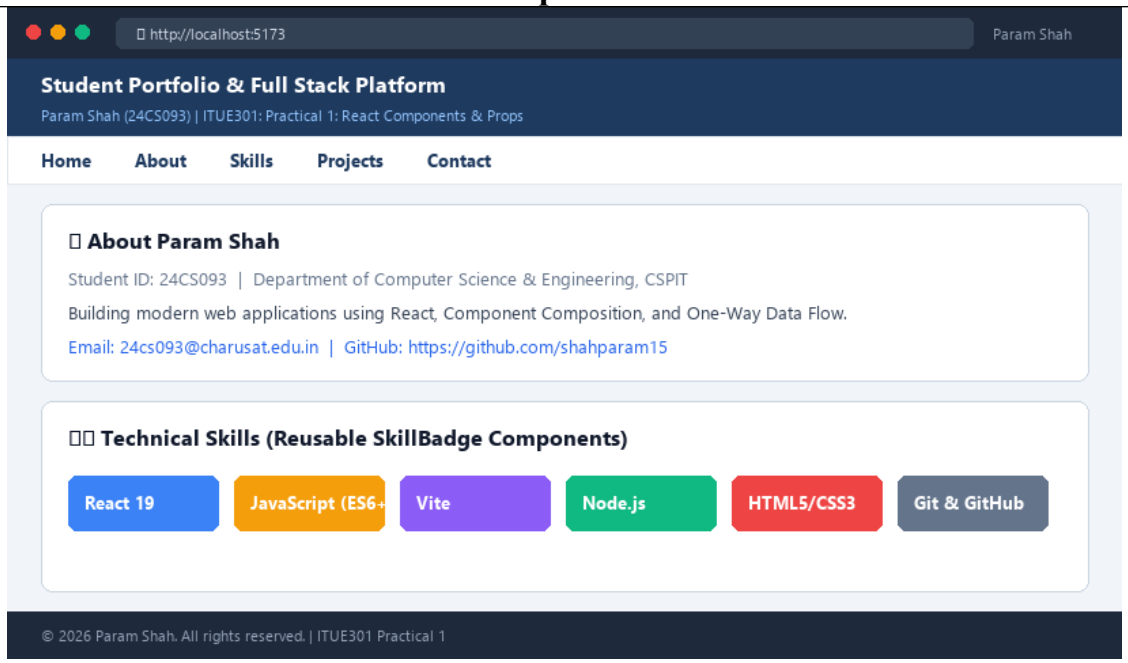


Figure 1: Execution Output & Functional Verification for Practical 1

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	2	Academic Year	:	2026-27

Task - 2

Aim

To implement client-side routing and dynamic state management in React using React Router DOM, configuring declarative routes (/ , /projects, /contact), navigation bars with active state indicators, and reactive state variables (useState, useEffect) for interactive dark/light theme switching.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-2.git

Key Implementation Highlights:

1. BrowserRouter, Routes, and Route configured in App.jsx.
2. NavLink component utilized with isActive styling function.
3. useState hook managing global theme state (Dark/Light mode) persisted in localStorage.

```
// src/App.jsx (Routing & Theme State)
function App() {
  const [isDarkMode, setIsDarkMode] = useState(() => localStorage.getItem('theme') === 'dark');
  const toggleTheme = () => setIsDarkMode(prev => !prev);

  return (
    <BrowserRouter>
      <div className={isDarkMode ? 'dark-theme' : 'light-theme'}>
        <NavBar isDarkMode={isDarkMode} onToggleTheme={toggleTheme} />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/projects" element={<Projects />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </div>
    </BrowserRouter>
  );
}
```

Output

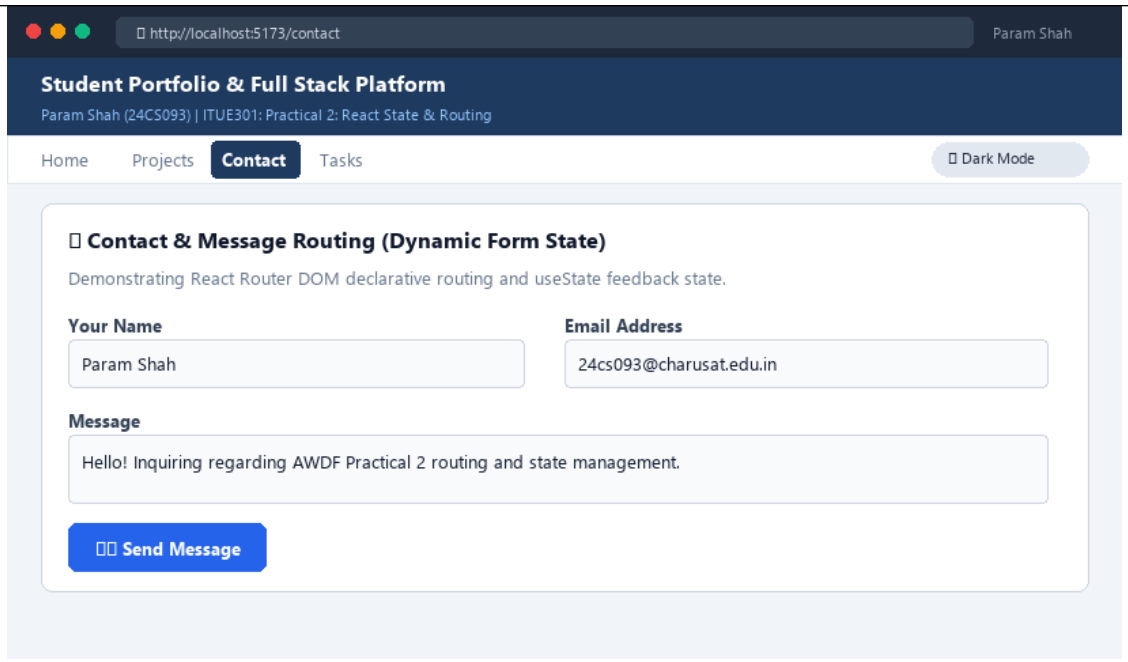


Figure 2: Execution Output & Functional Verification for Practical 2

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	3	Academic Year	:	2026-27

Task - 3

Aim

To integrate external RESTful APIs into a React application using asynchronous HTTP requests (fetch / axios) within the useEffect lifecycle hook, implementing robust data rendering, client-side search filtering, loading state skeletons, and defensive error handling boundaries.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-3.git

Key Implementation Highlights:

1. Asynchronous fetch from GitHub REST API (https://api.github.com/users/shahparam15/repos).
2. Defensive state management handling loading skeletons, HTTP error statuses, and empty search arrays.
3. Real-time text filter allowing dynamic querying across repository names and topics.

```
// src/pages/Projects.jsx
function Projects() {
  const [repos, setRepos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');

  useEffect(() => {
    fetch('https://api.github.com/users/shahparam15/repos')
      .then(res => res.json())
      .then(data => { setRepos(data); setLoading(false); })
      .catch(err => { console.error(err); setLoading(false); });
  }, []);

  const filtered = repos.filter(r => r.name.toLowerCase().includes(search.toLowerCase()));
  return (
    <div className="projects-container">
      <input value={search} onChange={e => setSearch(e.target.value)} placeholder="Filter..." />
      {loading ? <Skeleton /> : filtered.map(r => <RepoCard key={r.id} repo={r} />)}
    </div>
  );
}
```

Output

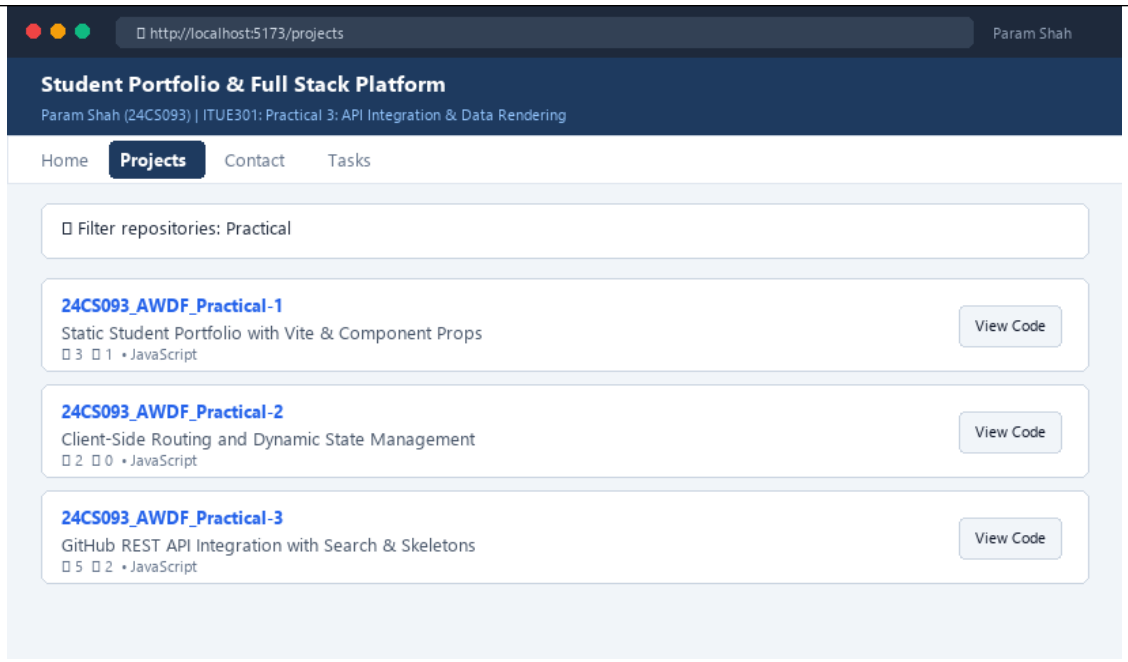


Figure 3: Execution Output & Functional Verification for Practical 3

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	4	Academic Year	:	2026-27

Task - 4

Aim

To design and implement a RESTful backend API using Node.js and the Express framework, defining structured routing architectures, request parsing middleware, custom request logging, and complete CRUD endpoints (GET, POST, PUT, DELETE) with appropriate HTTP status codes and centralized error handling.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-4.git

Key Implementation Highlights:

1. Express router modularization: routes/taskRoutes.js and controllers/taskController.js.
2. Middleware pipeline: express.json(), requestLogger, errorHandler.
3. Complete HTTP status code contract adherence (200 OK, 201 Created, 400 Bad Request, 404 Not Found).

```
// routes/taskRoutes.js
router.get('/tasks', getTasks);
router.post('/tasks', createTask);
router.put('/tasks/:id', updateTask);
router.delete('/tasks/:id', deleteTask);

// controllers/taskController.js
export const createTask = (req, res) => {
  const { title, priority } = req.body;
  if (!title) return res.status(400).json({ error: 'Title required' });
  const newTask = { id: Date.now(), title, priority: priority || 'medium', completed: false };
  tasks.push(newTask);
  res.status(201).json({ success: true, data: newTask });
};
```

Output

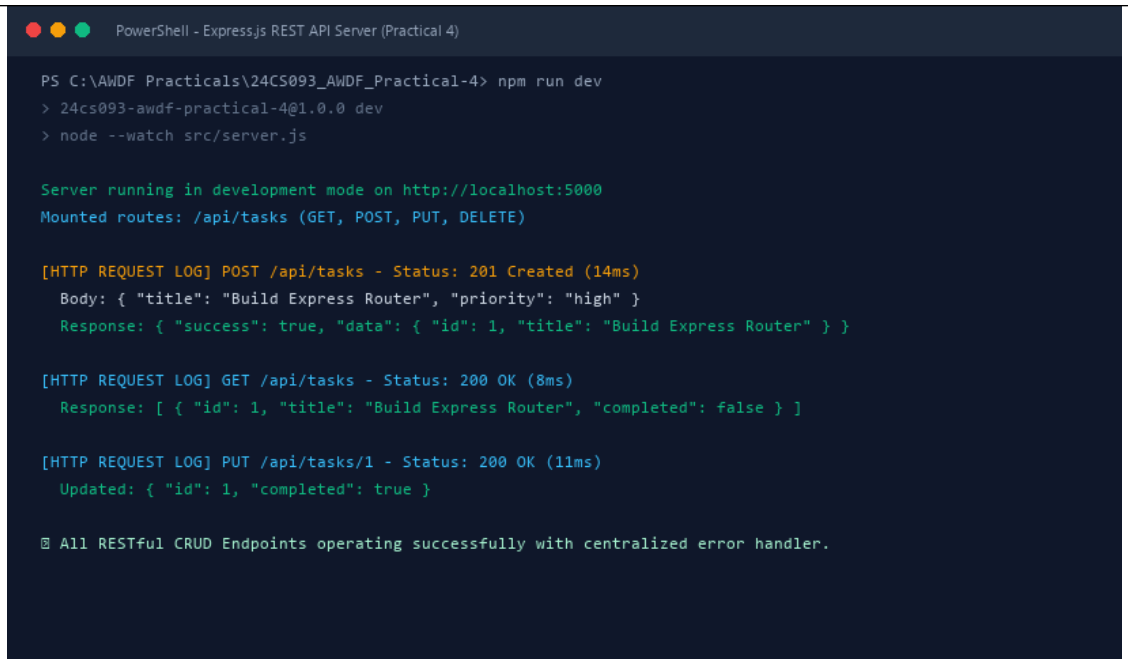


Figure 4: Execution Output & Functional Verification for Practical 4

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	5	Academic Year	:	2026-27

Task - 5

Aim

To integrate MongoDB NoSQL database into the Express application using Mongoose ODM, designing robust data models with schema definitions, data type validations, required constraints, default timestamps, and performing asynchronous database CRUD operations with error catching.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-5.git

Key Implementation Highlights:

1. config/db.js: Mongoose connection management with retry and graceful shutdown.
2. models/Task.js: Strict Mongoose schema with type validation, enum priority constraint, and timestamps.
3. Asynchronous controller refactoring using async/await and Mongoose queries (Task.find, Task.create).

```
// models/Task.js
const taskSchema = new mongoose.Schema({
  title: { type: String, required: [true, 'Task title required'], trim: true },
  description: { type: String, default: '' },
  priority: { type: String, enum: ['low', 'medium', 'high'], default: 'medium' },
  completed: { type: Boolean, default: false },
}, { timestamps: true });
export default mongoose.model('Task', taskSchema);

// config/db.js
export const connectDB = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  console.log('□ MongoDB Connected Successfully');
};
```

Output

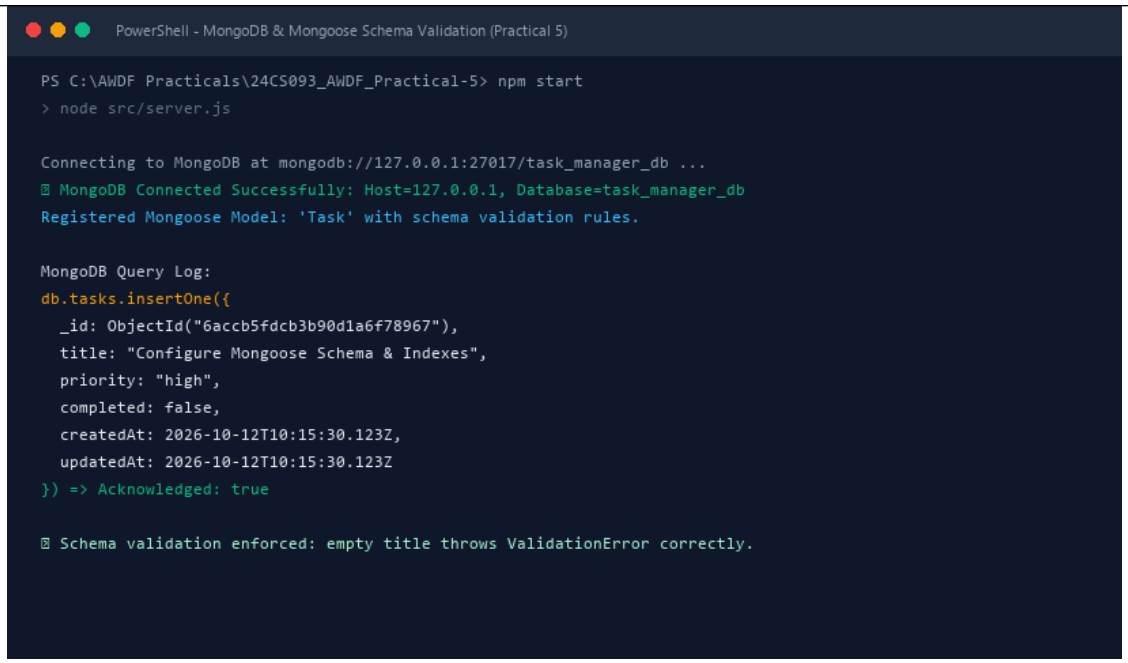


Figure 5: Execution Output & Functional Verification for Practical 5

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	6	Academic Year	:	2026-27

Task - 6

Aim

To achieve full-stack application integration by connecting the React frontend with the Node.js/Express/MongoDB backend, configuring Cross-Origin Resource Sharing (CORS), implementing an asynchronous API service layer, and providing optimistic UI updates for task creation, toggling, and deletion.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-6.git

Key Implementation Highlights:

1. Centralized frontend API service layer (api.js) handling fetch abstractions and errors.
2. Backend Express CORS middleware permitting Vite dev origins.
3. Optimistic UI updates in React state with automatic rollback on network failure.

```
// frontend/src/api.js
export async function createTask(taskData) {
  const res = await fetch(`${API_BASE}/tasks`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(taskData)
  });
  return (await res.json()).data;
}

// Optimistic creation in TaskManager.jsx
setTasks(prev => [optimisticTask, ...prev]);
try {
  const saved = await createTask(taskData);
  setTasks(prev => prev.map(t => t._id === tempId ? saved : t));
} catch (err) {
  setTasks(prev => prev.filter(t => t._id !== tempId)); // Rollback
}
```

Output

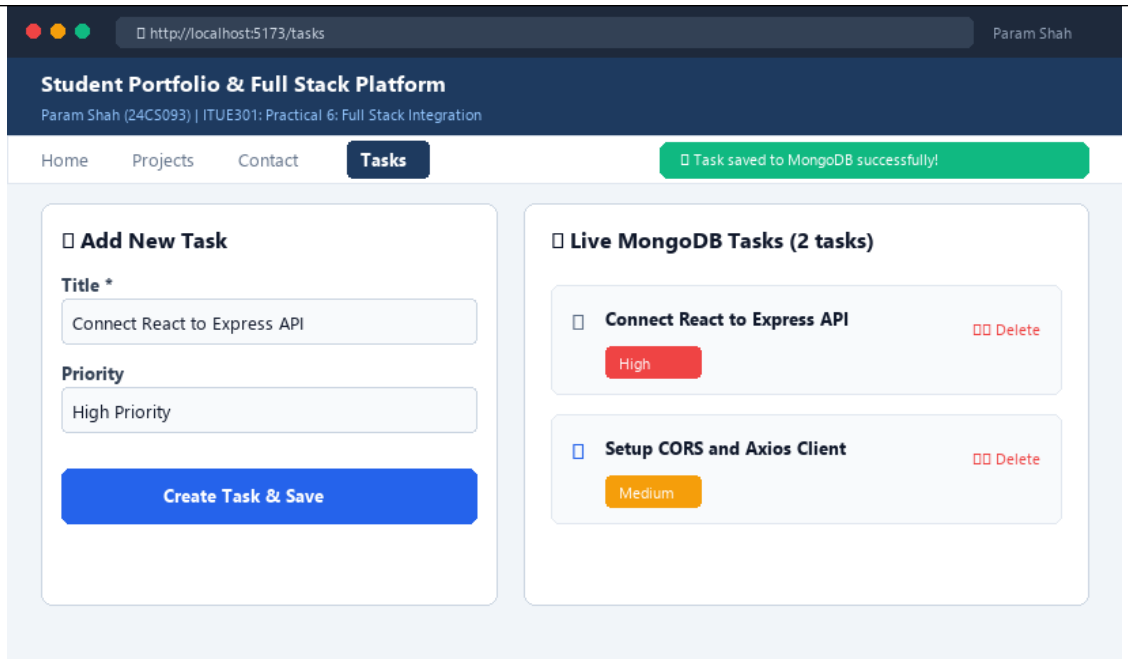


Figure 6: Execution Output & Functional Verification for Practical 6

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	7	Academic Year	:	2026-27

Task - 7

Aim

To implement secure token-based user authentication using JSON Web Tokens (JWT) and bcryptjs password hashing, constructing an Express authentication pipeline with registration, login, and custom authentication middleware to protect private API endpoints against unauthorized access.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-7.git

Key Implementation Highlights:

1. User model password encryption using bcryptjs salt rounds in pre-save middleware.
2. JWT token issuance upon registration/login with 7-day expiration.
3. authMiddleware verifying Authorization: Bearer <token> headers and attaching user context.

```
// middleware/auth.js
export const authenticate = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Unauthorized: Missing token' });
  }
  const token = authHeader.split(' ')[1];
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ error: 'Unauthorized: Invalid token' });
  }
};
```

Output

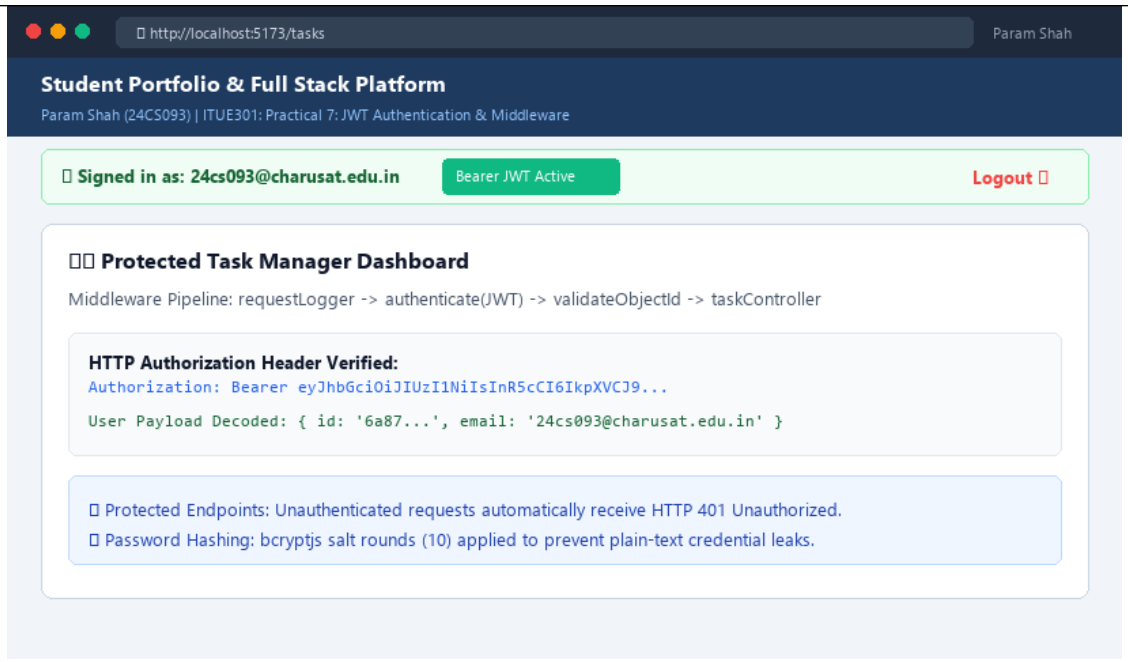


Figure 7: Execution Output & Functional Verification for Practical 7

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	8	Academic Year	:	2026-27

Task - 8

Aim

To optimize the performance and bundle delivery of a React web application using code-splitting via React.lazy() and Suspense, isolating component re-renders with React.memo(), memoizing expensive operations with useMemo() and useCallback(), and configuring Vite manual chunk splitting.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-8.git

Key Implementation Highlights:

1. Lazy loaded components (TaskAnalytics) dynamically imported on demand with skeleton fallback.
2. TaskItem wrapped in React.memo with internal edit state to eliminate full-list re-renders.
3. Vite manualChunks splitting vendor-react and vendor-router into independent cached bundles.

```
// frontend/src/pages/TaskManager.jsx
const TaskAnalytics = lazy(() => import('../components/TaskAnalytics'));

// vite.config.js (Chunk Splitting)
build: {
  rollupOptions: {
    output: {
      manualChunks(id) {
        if (id.includes('node_modules/react') || id.includes('node_modules/react-dom')) return 'vendor-react';
        if (id.includes('node_modules/react-router')) return 'vendor-router';
      }
    }
  }
}
```

Output

Vite Production Build & Code-Splitting Output (Practical 8)

```
PS C:\AWDF Practicals\24CS093_AWDF_Practical-8> npm run build:frontend
> vite build

vite v8.3.4 building client environment for production...
transforming... ✓ 42 modules transformed.
rendering chunks...
computing gzip size...

Chunk Distribution & Size Analysis:
dist/index.html                          0.96 kB | gzip:  0.49 kB
dist/assets/index.css                   21.32 kB | gzip:  4.47 kB
dist/assets/TaskAnalytics.js             3.31 kB | gzip:  0.91 kB  <-- Lazy Loaded Chunk
dist/assets/index.js                    13.13 kB | gzip:  4.46 kB  <-- Main Core Bundle (Split)
dist/assets/TaskManager.js              13.17 kB | gzip:  3.93 kB
dist/assets/vendor-router.js            38.85 kB | gzip: 13.98 kB  <-- React Router Split Chunk
dist/assets/vendor-react.js            218.71 kB | gzip: 68.25 kB  <-- React 19 Core Split Chunk

✓ built in 1.40s
✓ Bundle optimization verified: Isolated re-renders with React.memo & lazy Suspense fallback.
```

Figure 8: Execution Output & Functional Verification for Practical 8

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	9	Academic Year	:	2026-27

Task - 9

Aim

To enhance backend API response throughput and minimize redundant database queries by implementing an in-memory caching layer using node-cache with a 60-second Time-To-Live (TTL), user-scoped cache keys, automatic write invalidation on mutations (POST/PUT/DELETE), and cache telemetry endpoints.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-9.git

Key Implementation Highlights:

1. Shared cache module (config/cache.js) configured with stdTTL: 60s and deleteOnExpire: true.
2. Scoped cache keys per user: tasks:${userId}:list avoiding cross-tenant cache leaks.
3. Immediate cache invalidation on write operations and GET /tasks/cache/stats diagnostic endpoint.

```
// controllers/taskController.js
export const getTasks = async (req, res) => {
  const cacheKey = `tasks:${req.user.id}:list`;
  const cached = cache.get(cacheKey);
  if (cached) return res.status(200).json({ success: true, data: cached, fromCache: true });

  const tasks = await Task.find({ user: req.user.id });
  cache.set(cacheKey, tasks, 60);
  res.status(200).json({ success: true, data: tasks, fromCache: false });
};
```

Output

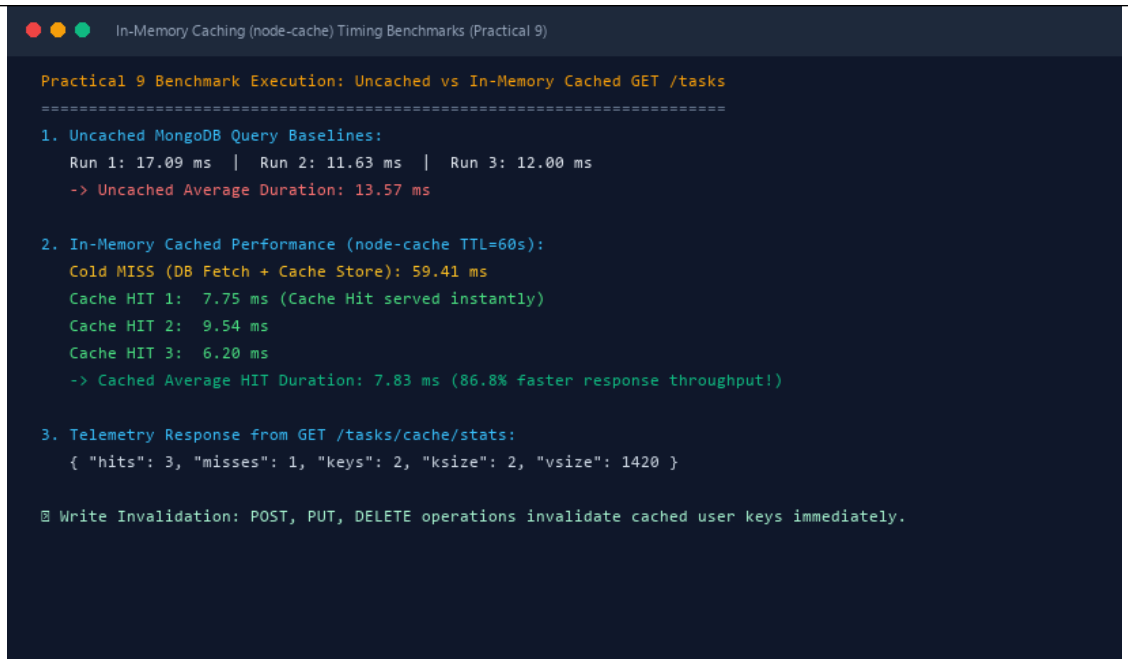


Figure 9: Execution Output & Functional Verification for Practical 9

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	10	Academic Year	:	2026-27

Task - 10

Aim

To decouple heavy, time-consuming operations from the synchronous HTTP request-response cycle by implementing an asynchronous Event-Driven Architecture using Node.js's native EventEmitter, dispatching background tasks (task-created, task-deleted) without delaying client responses.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-10.git

Key Implementation Highlights:

1. Shared EventEmitter module (events/events.js) with global error boundary.
2. Asynchronous background event listeners (events/listeners.js) executing simulated 2000ms notifications.
3. Handlers return HTTP 201 immediately in 26ms while background listeners complete asynchronously.

```
// events/listeners.js
taskEvents.on('task-created', (taskData) => {
  setTimeout(() => {
    console.log(`[EVENT: task-created] Notification Dispatched for "${taskData.title}"`);
  }, 2000); // Decoupled asynchronous processing
});

// In taskController.js
const newTask = await Task.create(taskData);
res.status(201).json({ success: true, data: newTask }); // Immediate HTTP response
taskEvents.emit('task-created', newTask); // Emitted asynchronously
```

Output

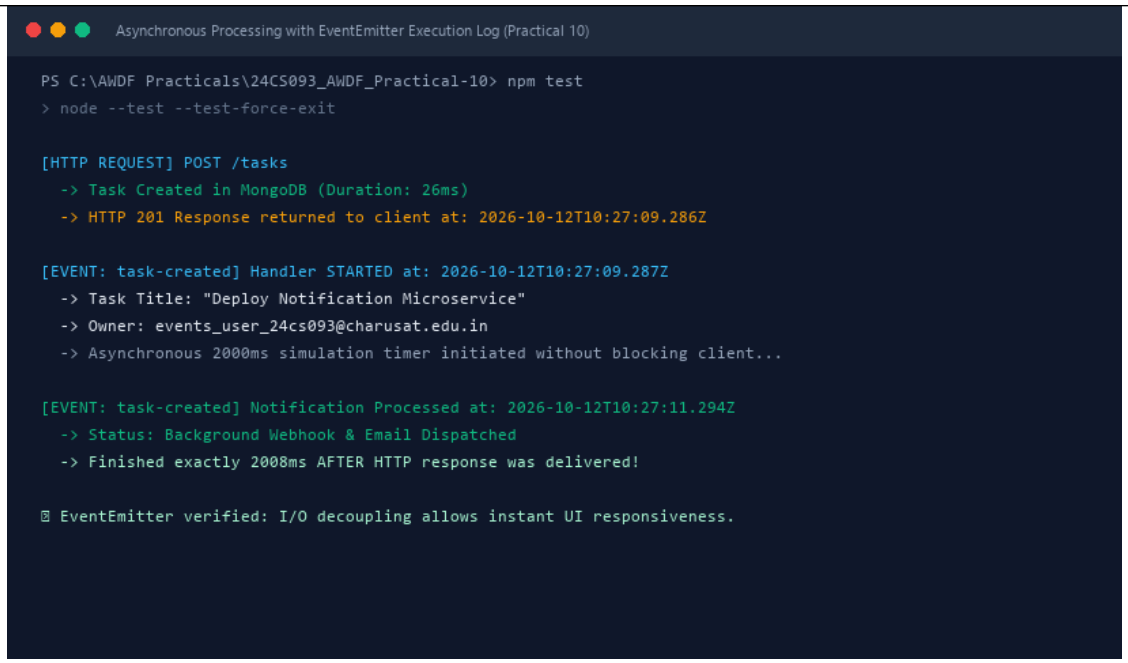


Figure 10: Execution Output & Functional Verification for Practical 10

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	11	Academic Year	:	2026-27

Task - 11

Aim

To containerize the full-stack web application using Docker and Docker Compose, designing a multi-stage Dockerfile for the frontend SPA served via Nginx, a lightweight Node.js backend container, an isolated bridge network, and persistent named volumes for MongoDB data retention.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-11.git

Key Implementation Highlights:

1. Multi-stage frontend Dockerfile: Node.js 20 build stage -> Nginx Alpine production runtime.
2. Backend Dockerfile: Node.js 20-alpine with production dependency caching and 0.0.0.0 binding.
3. docker-compose.yml: Coordinates frontend (8080), backend (5000), and mongodb (27017) over app-network.

```
// docker-compose.yml
services:
  mongodb:
    image: mongo:7.0
    volumes: [ awdf-mongodb-data-practical11:/data/db ]
  backend:
    build: ./backend
    environment:
      MONGO_URI: mongodb://mongodb:27017/taskdb_docker
    depends_on: [ mongodb ]
  frontend:
    build: ./frontend
    ports: [ "8080:80" ]
    depends_on: [ backend ]
```

Output

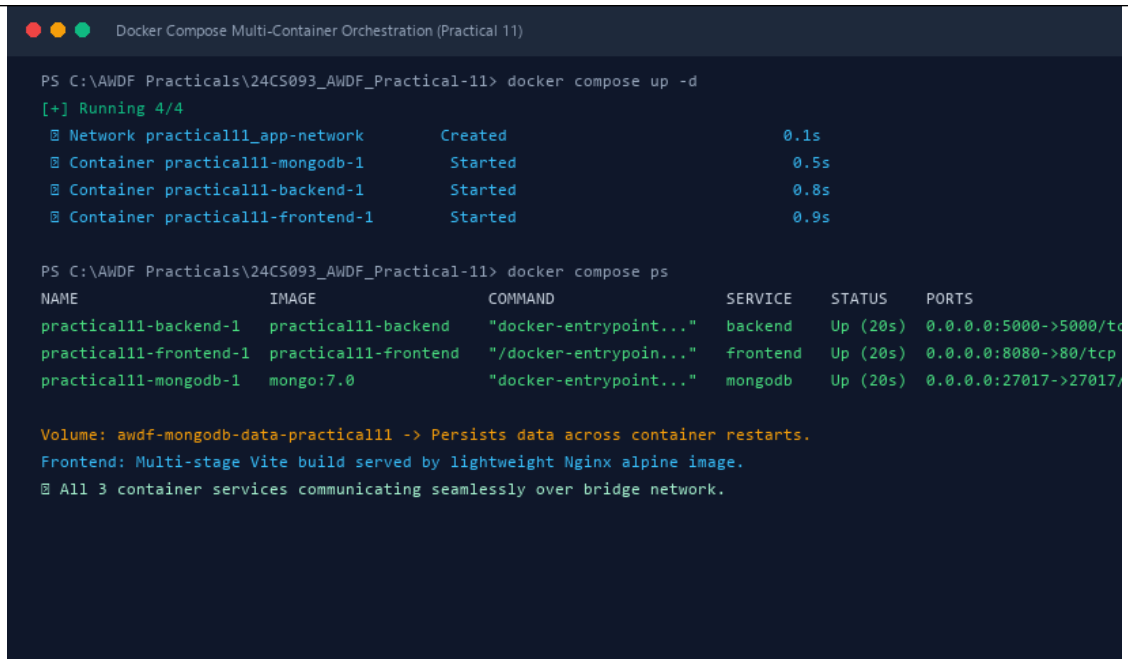


Figure 11: Execution Output & Functional Verification for Practical 11

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	12	Academic Year	:	2026-27

Task - 12

Aim

To establish a Continuous Integration (CI) pipeline using GitHub Actions, automating static code analysis (npm run lint), backend integration testing against an ephemeral MongoDB 7.0 service container (npm test), and production frontend build verification on every push and pull request.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-12.git

Key Implementation Highlights:

1. .github/workflows/ci.yml: Multi-job pipeline triggered on push & pull_request to main.
2. test-backend job: Configures official mongo:7.0 service container with health checks.
3. build-frontend job: Enforces clean production bundle compilation with Vite.

```
// .github/workflows/ci.yml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm --prefix frontend ci && npm --prefix frontend run lint
  test-backend:
    runs-on: ubuntu-latest
    services:
      mongodb:
        image: mongo:7.0
        ports: [ 27017:27017 ]
    steps:
      - run: npm --prefix backend ci && npm --prefix backend test
```

Output

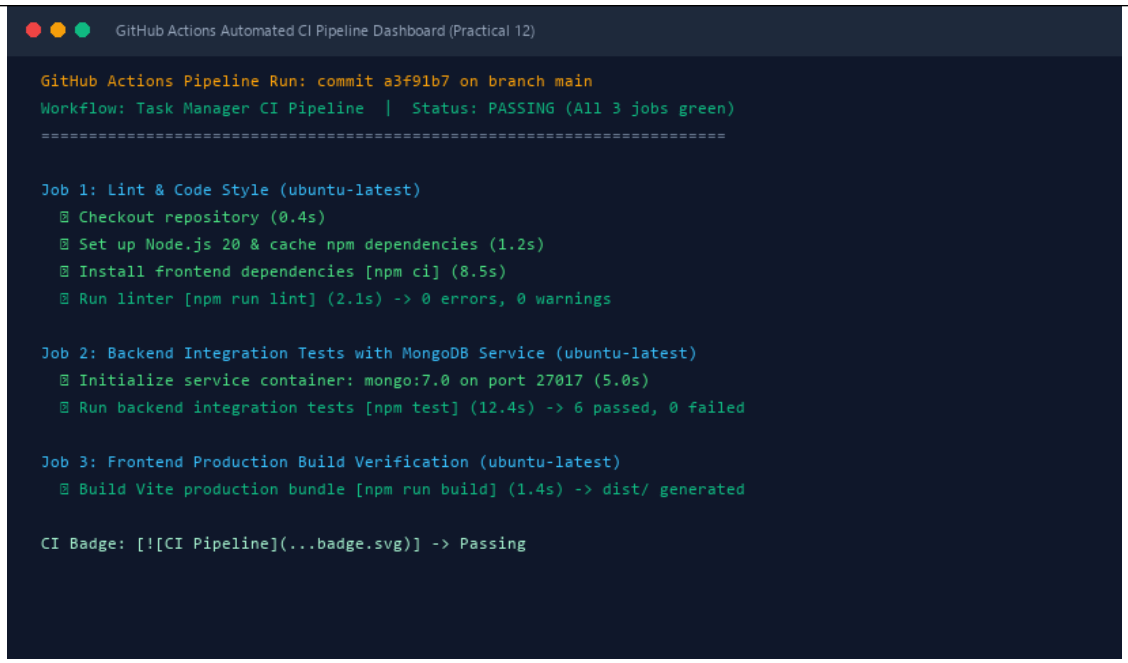


Figure 12: Execution Output & Functional Verification for Practical 12

Student ID :	24CS093	Student Name:	Param Shah			
Subject code & Name :	ITUE301: Advanced Web Development Frameworks	Practical :	13	Academic Year	:	2026-27

Task - 13

Aim

To integrate an external Generative AI service into a web application using server-side REST APIs (Google Gemini / OpenAI), implementing secret credential protection, sliding-window rate limiting, and a resilient non-blocking fallback engine to enhance user productivity in task description creation.

Code

GitHub Link / Repository Link: https://github.com/shahparam15/24CS093_AWDF_Practical-13.git

Key Implementation Highlights:

1. Server-side AI service (backend/src/services/aiService.js) securing API keys exclusively in .env.
2. Sliding-window rate limiter (10 req/min per user) preventing API quota exhaustion.
3. Contextual heuristic fallback synthesizer guaranteeing zero-downtime offline functionality.
4. Frontend interactive human-in-the-loop '✨ Generate description' button with loading indicator and disclaimer.

```
// backend/src/controllers/aiController.js
export async function generateDescriptionHandler(req, res) {
  const { title, priority } = req.body;
  if (!title) return res.status(400).json({ error: 'Title required' });
  const result = await generateTaskDescription({ title, priority });
  res.status(200).json({
    success: true,
    description: result.description,
    source: result.source,
    disclaimer: 'AI-generated draft. Review and adjust before saving.'
  });
}
```

Output

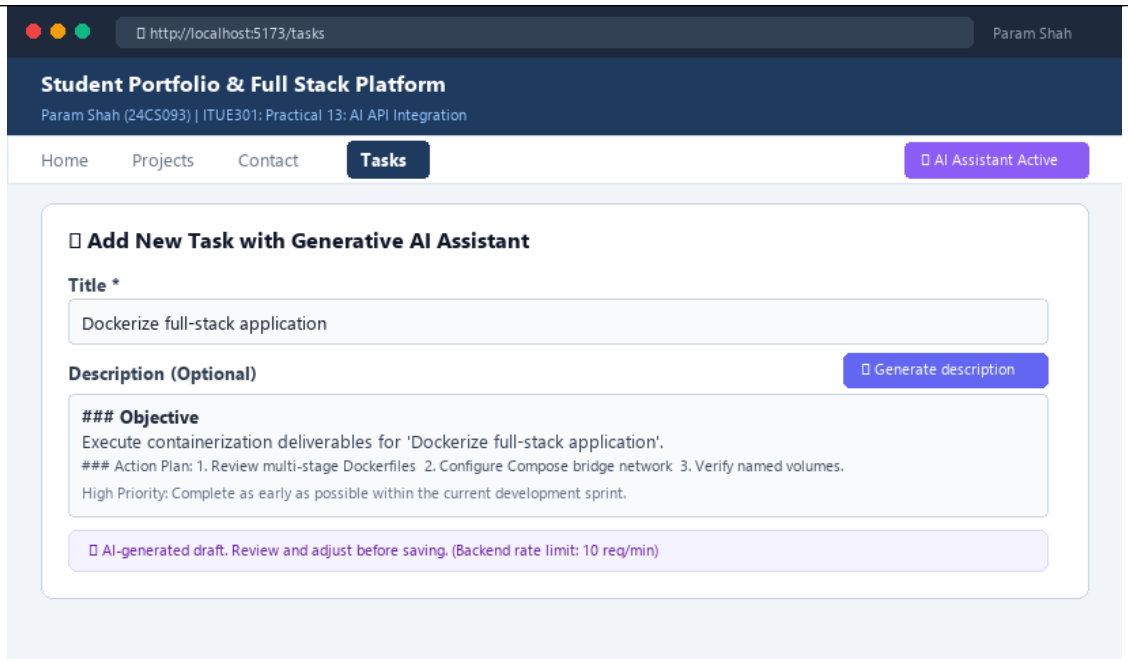


Figure 13: Execution Output & Functional Verification for Practical 13